

Teil 3:

Architektur und Modellierung

Prof. Dr. Falk Howar

Automated Quality Assurance
Lehrstuhl für Software Engineering
Fakultät für Informatik
TU Dortmund



Modellierung / Meta-Modellierung.

- Konzepte und Methoden im Bereich der Modellierungssprachen **aufzählen** und **erläutern**



- Verhalten einfacher Komponenten in Modellen **spezifizieren**



- (Formale) Methoden zur Ausführung und Komposition von Modellen einfacher Komponenten **erläutern** und **anwenden**



Analyse von Modellen.

- Bounded Model Checking und Invariantenbeweise zur Überprüfung von Eigenschaften von Komponenten **anwenden**



Agenda

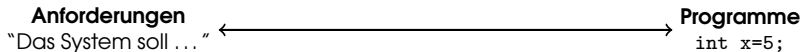
1. Motivation
2. Begriffe und Definitionen
3. Modellierung des Verhaltens von Komponenten
4. Analyse des Verhaltens von Komponenten
5. Literatur und Standards

Werkzeugbau im Software Engineering



<https://flic.kr/p/fkmB7T>, (CC BY 2.0)

Semantische Lücke



Fachlichkeit vs. Technik

- Abstraktionsniveau derzeitiger Entwicklungsansätze zu niedrig
- Umsetzung fachlicher Basiskonzepte dominiert durch Technik
- Anwendungsentwickler benötigen umfangreiches technisches Wissen statt sich auf fachliche Anwendungsdomäne zu konzentrieren

Fehlende Durchgängigkeit

- Fehlender Zusammenhang zwischen Anforderungen, Dokumentation und Code
- Dokumentation nicht aktualisiert
→ nahezu nutzlos.
- Umsetzung von Anforderungen nicht nachvollziehbar
→ Abnahme schwierig

Semantische Lücke (contd.)



Fachlichkeit vs. Technik

Bedarf nach höherem
Abstraktionsniveau

→

Modelle Abstrahieren und fokussieren
auf das Wesentliche

Fehlende Durchgängigkeit

Durchgängigere Auswahl der
Ausdrucksmittel

→

Semantische Brücke von fachlicher
Problemwelt in technische Lösungswelt

Von Anforderungen zu Programmen

Anforderungen

Integration von
Aspekten

Modellierungssprachen

Konkretisierung von
Lösungen



Modellgetriebene Softwareentwicklung

Aspekte von
Problemen

Modellierungssprachen



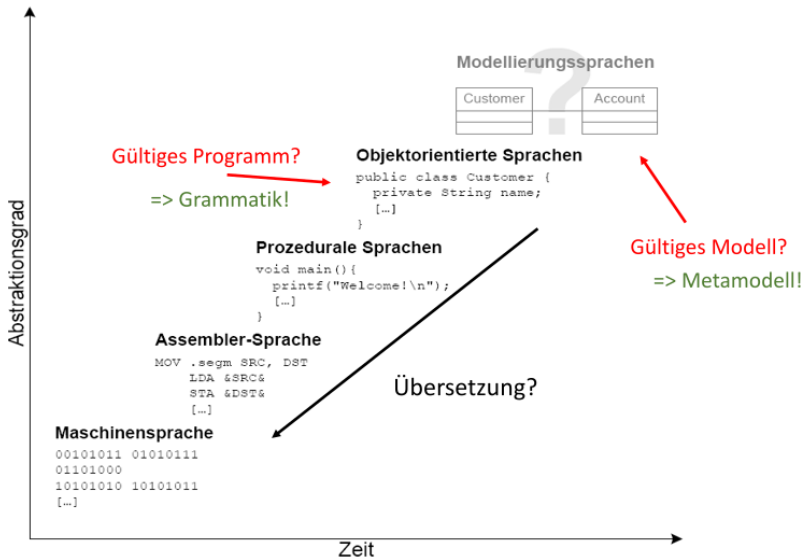
Domänenspezifische Sprachen

Abstraktion von
Implementierungsdetails

Programme



Entwicklung von Modellierungssprachen



Korrektheit von Modellen

Syntaktisch

- Ist es ein gültiges Modell?
- Ist das Modell korrekt gezeichnet?

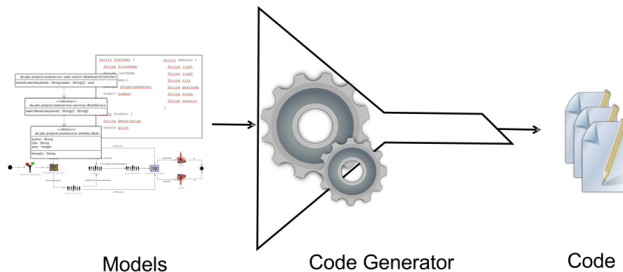
Voraussetzung für Übersetzung

Semantisch

- Ist das Modell korrekt?
- Erfüllt das Modell die Anforderungen?

Voraussetzung für Verwendung

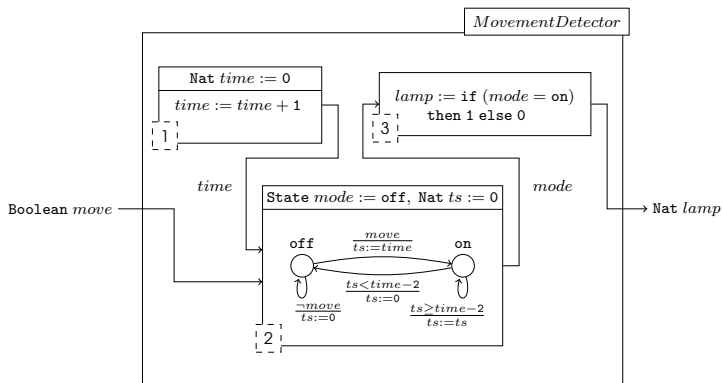
Übersetzung von Modellen



Trennung fachlicher und technischer Aspekte:

- Fachliche Lösung im Modell
- Technische Aspekte der Lösung im Code-Generator

Beispiel: Remote Home Control



Beispiele für Fragen:

- Ist das ein gültiges Modell?
- Geht das Licht immer an, wenn sich etwas bewegt?
- Meta: Wie stellt man fest, ob das Modell gültig ist?

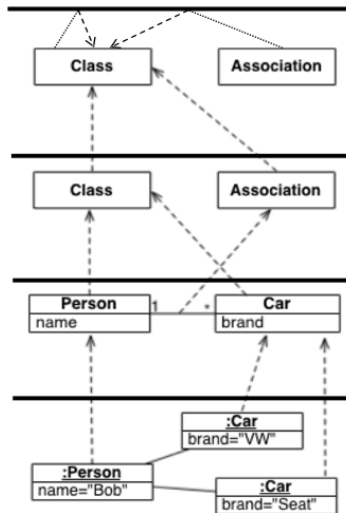
Modellierung

Ziele:

- Trennung von fachlicher und technischer Lösung
- Konsistente Ausdrucksmittel von den Anforderungen bis zum Code
- Methoden zum Entwickeln von Modellierungssprachen

Fragen:

- Wie kann man entscheiden ob ein Modell gültig und korrekt ist?
- Was ist die Bedeutung eines Modells?
- Wie kann man aus Modellen Code generieren?



Agenda

1. Motivation
2. Begriffe und Definitionen
3. Modellierung des Verhaltens von Komponenten
4. Analyse des Verhaltens von Komponenten
5. Literatur und Standards

Definition (Modell)

Eine Modell ist eine vereinfachende Beschreibung der Realität. Modelle haben drei Eigenschaften [Stachowiak]:

- ① **Abbild:** Ein Modell ist immer ein Modell von etwas (von einem Original), das es repräsentiert.
- ② **Verkürzung:** Ein Modell bildet nur einige Aspekte des Originals ab.
- ③ **Nützlich:** Ein Modell kann für einen bestimmten Zweck anstelle des Originals benutzt werden.

Aspekte, die typischerweise modelliert werden:

- Struktur
- Beziehungen
- Verhalten

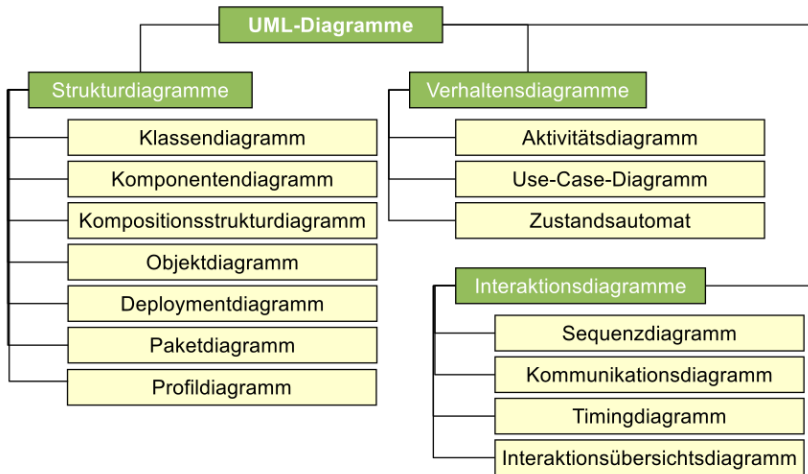
Definition (Modellierungssprache)

Eine Modellierungssprache ist eine Sprache, in der Modelle ausgedrückt werden.

Beispiel: Unified Modeling Language (UML)

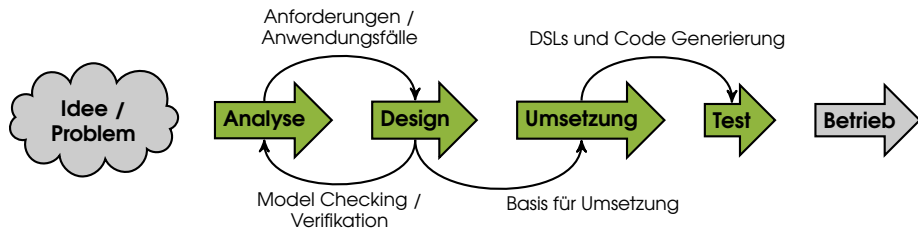
- Modellierung, Dokumentation, Spezifizierung und Visualisierung von komplexen Softwaresystemen.
- Standard der Object Management Group (OMG).
- Unterschiedliche Modellierungskonzepte auf einheitlicher Basis:
 - 14 Diagrammtypen in Version 2.4.1
- Modell vs. Diagramm:
 - UML-Modell besteht aus einem oder mehreren Diagrammen.
 - Ein Diagramm entspricht einer bestimmten Sicht ("View") auf Modell (vgl. IEEE 1471).

Wiederholung: Arten von UML-Diagrammen



Frage: Woher wissen Sie, wie man diese Diagramme nutzt?

Einordnung



Querschnittsthema: Modellierungssprachen
(Methoden des Werkzeugbaus)

Definition (Metamodell)

Für eine Klasse von Modellen beschreibt ein Metamodell die Modellierungssprache, in der die Modelle formuliert ist. Ein Metamodell besteht aus

- abstrakter Syntax (Modell-Elemente und deren mögliche Beziehungen),
- konkreter Syntax (Darstellung von Modell-Elementen in Diagrammen), und
- Semantik (Festlegung der Bedeutung von Modellen).

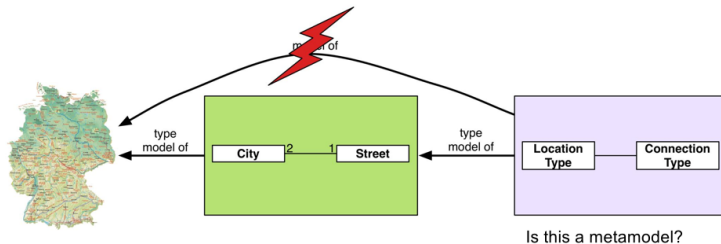
“Meta-“, griechisch: $\mu\epsilon\tau\alpha$: nach bzw. hinter

Beispiele für Bedeutung von “Meta-“:

[Kühne]: “meta is used whenever an operation is applied twice”, z.B.:

- Meta-Diskussion: eine Diskussion darüber wie man diskutiert
- Meta-Lernen: Erlernen von Strategien zum Lernen

Metamodell: Modell eines Modells

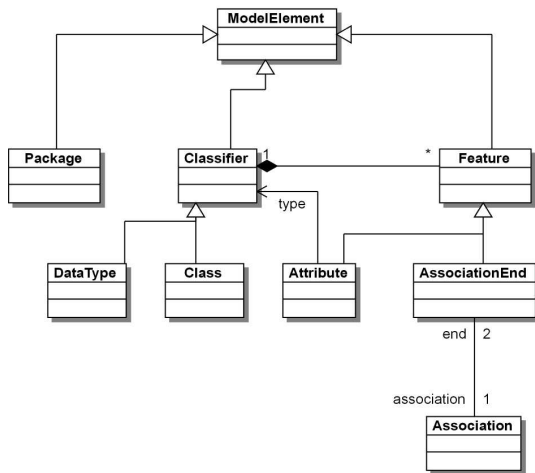


Yes, as we apply the same non-transitive relation “type model of” twice!
→ anti-transitivity through double classification

- ① “[...] any relation between two entities, which is going to be used to build up a meta-entity **must not be transitive.**”
- ② “[...] in order to create metamodell we need **the same nontransitive relationship [...]** **twice.**”

Meta Object Facility (MOF)

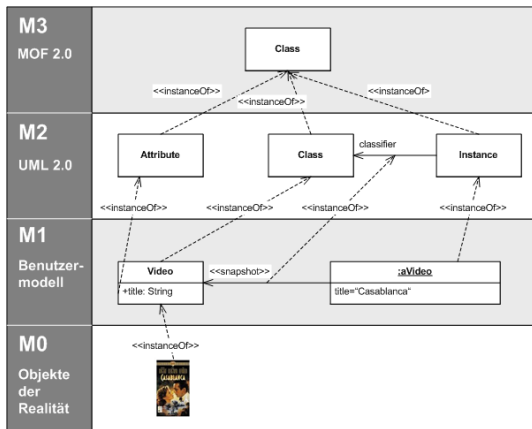
MOF Metamodel (vereinfacht)



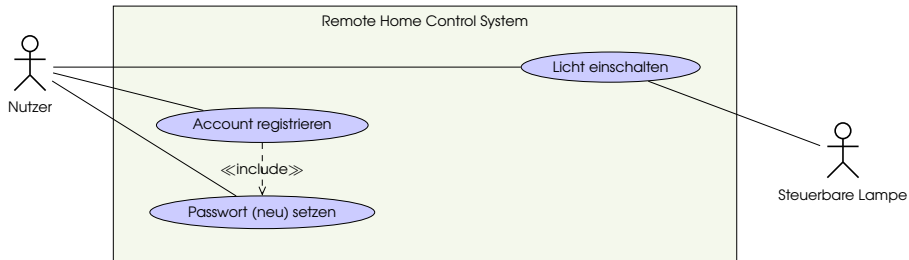
- Standard MOF 2.4.2
ISO/IEC 19508:2014
Information technology -
Meta Object Facility (MOF)
- Meta-Metamodell für UML
- Meta-Meta:
Beschreibt, wie man
Metamodelle (=Elemente
und Struktur von Modellen)
beschreibt
- Selbstreferenzierendes
Metamodell:
beschreibt sich selbst

Meta Object Facility (MOF)

- M3: Meta-Metamodell (MOF-Ebene, beschreibt, wie Metamodelle aufgebaut und strukturiert sind)
- M2: Metamodell (Definiert, wie Modelle aufgebaut und strukturiert sind)
- M1: Modell (Beschreibt Realität)
- M0: Konkret (Instanziiert Modell)



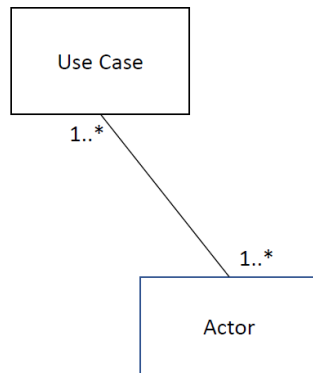
Beispiel: Anwendungsfalldiagramm



Abstrakte Syntax

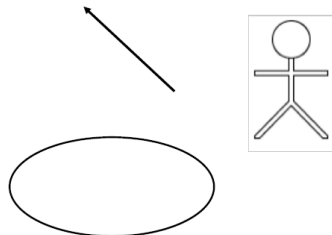
Beispiel: UML Anwendungsfalldiagramm

- Definiert Struktur der Symbole durch Konzepte, Beziehungen zwischen Konzepten und Integrationsbedingungen
- Erfasst nur Modellierungskonzepte (Abstraktion)
- Definition der abstrakten Syntax als:
 - Graphische Sprache: Metamodelle oder Graphgrammatiken
 - Textuelle Sprache: (Baum-)Grammatiken



- Konkrete Syntax definiert Symbole für Modellierungssprache
- Beispiele:
 - Textuelle Sprache
 - Graphische Sprache
 - Hybride (textuell und graphisch)

Beispiel: UML Anwendungsfalldiagramm

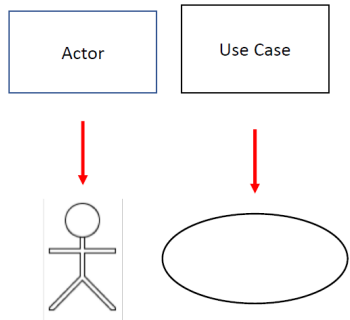


Beziehung Konkrete Syntax und Abstrakte Syntax

Beispiel: UML Anwendungsfalldiagramm

Syntaktische Abbildung:

Beziehung zwischen konkreter und abstrakter Syntax:



Ausnutzen von Meta-Informationen

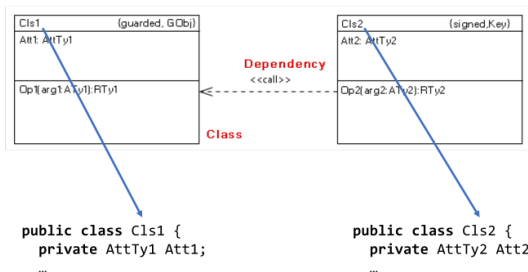
- **Gültigkeitsprüfung:**

Z.B.: darf ein **Element** vom **Typ X** mit einem Element vom Typ Y verbunden werden?

- **Übersetzung:**

Z.B. für jedes **Element** vom **Typ X** ...

UML Klassendiagramm



- Für alle Element vom Typ "Klassen" ...

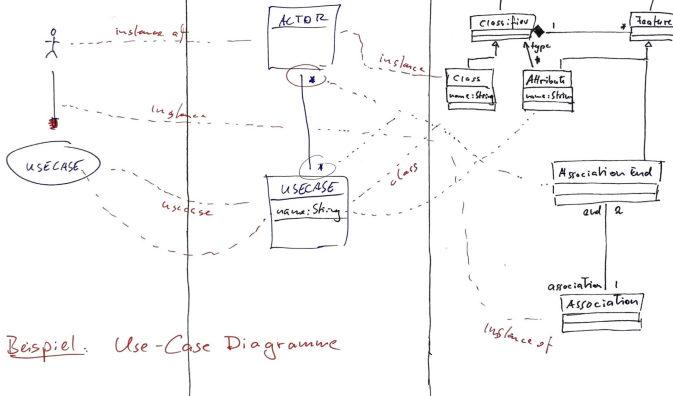
- Für alle Elemente vom Typ "Attribut" ...

Beispiel: Modell, Metamodell und Meta-Metamodell

Modell
(in UML Notation Syntax)

Metamodell
(in Class Diagram Syntax)

NOT (von einfach)



Beispiel: Use-Case Diagramme

Zusammenfassung: Meta-Modellierung

- Metamodelle beschreiben Modelle
- Metamodelle werden benötigt, um Modelle automatisiert zu analysieren, zu bearbeiten oder zu übersetzen
- MOF ist das Metamodell hinter UML
- Vier Ebenen M0 bis M3 (Reale Objekte, Modelle, Metamodelle, Metametamodelle)
- Abstrakte Syntax: Objekte und Beziehungen im Modell
- Konkrete Syntax: Darstellung im Modell

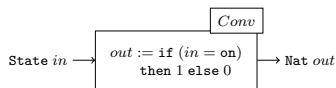
Agenda

1. Motivation
2. Begriffe und Definitionen
3. Modellierung des Verhaltens von Komponenten
4. Analyse des Verhaltens von Komponenten
5. Literatur und Standards

Modellierung des Verhaltens von Komponenten

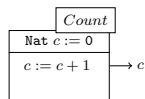
Ziel 1:

- Konzepte von Meta-Modellierung und Domänen-Spezifischen Sprachen am Beispiel verdeutlichen



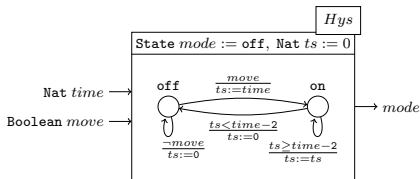
Ziel 2:

- Formale Spezifikation für Komponenten und Komposition (Anknüpfend an Vorlesung über Architektur)



Ziel 3:

- Basis für Model Checking und Invariantenbeweise



Wdh. Variablen / Typen

Typen:

- `Nat`: Ganze Zahlen
- `Boolean`: Boolesche Werte $\{0, 1\}$
- Aufzählungen. Z.B. Typ `State` mit Werten $\{\text{on}, \text{off}\}$

Variablen:

- Menge X von getypten Variablen

Wdh. Getypte Ausdrücke

Syntax

Für einen fixen Aufzählungstyp, z.B. **State**:

$e \in \mathbf{EExp}$ (Aufzählungs-Ausdrücke)

$e ::= z \mid c$

$z \in X$ Variable vom Aufzählungstyp **State**

$c \in \mathbf{State}$ konkreter Wert vom passenden Typ

Erweiterung: Getypte Ausdrücke

Syntax

$a \in \mathbf{AExp}$ (Arithmetische Ausdrücke)

$a ::= x \mid \mathbf{n} \mid (a_1 \text{ } op_a \text{ } a_2) \mid \text{if } b \text{ then } a_1 \text{ else } a_2$

$b \in \mathbf{BExp}$ (Boolesche Ausdrücke)

$b ::= y \mid \mathbf{true} \mid \mathbf{false} \mid \neg b \mid (b_1 \text{ } op_b \text{ } b_2) \mid (a_1 \text{ } op_r \text{ } a_2) \mid (z = c)$

$x \in X$ Variable Typ $\mathbf{Nat}$

$y \in X$ Variable Typ $\mathbf{Boolean}$

$z \in X$ Variable Typ $\mathbf{State}$

$\mathbf{n}$ Literal für $n \in \mathbb{Z}$

$c \in \mathbf{State}$

$op_a \in \mathbf{Op_a}$ arithmetische Op.: $+$, $-$

$op_b \in \mathbf{Op_b}$ Boolesche Op.: $\wedge$, $\vee$

$op_r \in \mathbf{Op_r}$ Relationale Op.: $>$, $<$, $=$

Wdh. Auswertung von Ausdrücken

Semantik

- Set von Variablen X
- Ausdrücke aus **Exp** über Variablen aus X
- Fixer Aufzählungstyp z.B., State mit Werten $\{\text{on}, \text{off}\}$

Auswertung:

- Funktion $\alpha_X : \mathbf{AExp} \times V_X \mapsto \mathbb{Z}$
- Funktion $\beta_X : \mathbf{BExp} \times V_X \mapsto \{0, 1\}$
- Funktion $\gamma_X : \mathbf{EExp} \times V_X \mapsto \mathbf{State}$
- Belegung v von X

Erweiterung: Auswertung von Ausdrücken

Semantik

Arithmetische Ausdrücke:

- $\alpha_X[\text{if } b \text{ then } a_1 \text{ else } a_2, v] = \alpha_X[a_1, v] \text{ wenn } \beta_X[b, v] = 1 \text{ sonst } \alpha_X[a_2, v]$

Umgang mit Variablen

Ausdrücke mit Variablen:

- Wir schreiben $\varphi[\mathbf{x}]$ um anzuzeigen, dass der Ausdruck φ über einer Menge $\mathbf{x}$ von Variablen formuliert ist
- Mit $\mathbb{F}_{\mathbf{x}}$ bezeichnen wir Menge aller möglichen logischen Formeln aus **Exp** mit Variablen $\mathbf{x}$. Wenn $\varphi[\mathbf{y}] \in \mathbb{F}_{\mathbf{x}}$ dann $\mathbf{y} \subseteq \mathbf{x}$.

Apostrophierte Variablen und Ersetzung in Termen und Belegungen:

- Für $\mathbf{x} = \{x_1, \dots, x_k\}$ sei $\mathbf{x}' = \{x'_1, \dots, x'_k\}$ und $\mathbf{x}^i = \{x_1^i, \dots, x_k^i\}$
- Wir schreiben $\varphi[\mathbf{x}'/\mathbf{x}]$ für den Ausdruck, der aus φ durch Ersetzen der Variablen in $\mathbf{x}$ durch jene in $\mathbf{x}'$ entsteht
- Wir schreiben $v[\mathbf{x}'/\mathbf{x}]$ für die Belegung, die aus v durch Ersetzen der Variablen in $\mathbf{x}$ durch jene in $\mathbf{x}'$ entsteht

Beispiel:

- Für $\mathbf{x} = \{x_1, x_2\}$ und $\varphi = ((x_2 = \text{on}) \wedge (x_1 > 50))$ und $v_{\mathbf{x}} = \{x_1 \mapsto 1, x_2 \mapsto \text{off}\}$
- ist $\varphi[\mathbf{x}'/\mathbf{x}] = ((x'_2 = \text{on}) \wedge (x'_1 > 50))$ und $v_{\mathbf{x}}[\mathbf{x}'/\mathbf{x}] = \{x'_1 \mapsto 1, x'_2 \mapsto \text{off}\}$

Belegungen, Modelle und Erfüllbarkeit

Belegung:

- $v_{\mathbf{x}} : \mathbf{x} \mapsto \mathbb{N} \cup \{0, 1\} \cup \{\text{on}, \text{off}\}$ ist eine typ-korrekte Belegung der Variablen in $\mathbf{x}$
- $V_{\mathbf{x}}$ ist die Menge aller möglichen typ-korrekten Belegungen der Variablen in $\mathbf{x}$
- Für $\mathbf{x} \cap \mathbf{y} = \emptyset$ sei $v_{\mathbf{x}} \circ v_{\mathbf{y}}$ eine Belegung von $\mathbf{x} \cup \mathbf{y}$ mit

$$v_{\mathbf{x}} \circ v_{\mathbf{y}}(x) =_{def} \begin{cases} v_{\mathbf{x}}(x) & \text{für } x \in \mathbf{x} \\ v_{\mathbf{y}}(x) & \text{für } x \in \mathbf{y} \end{cases}$$

Modell:

- Belegung $v_{\mathbf{x}}$ ist ein Modell für $\varphi[\mathbf{x}] \in \mathbf{BExp}$ wenn $\beta_{\mathbf{x}}[\varphi, v_{\mathbf{x}}] = 1$
- Wir schreiben $v_{\mathbf{x}} \models \varphi[\mathbf{x}]$

Erfüllbarkeit:

- Ausdruck $\varphi[\mathbf{x}] \in \mathbf{BExp}$ ist erfüllbar wenn ein Modell für $\varphi[\mathbf{x}]$ existiert

Definition (Synchrone Reaktive Komponente)

Eine Synchrone Reaktive Komponente C ist ein Tupel $\langle i, s, o, I, \Gamma \rangle$ mit

- einer Menge i von Eingabevariablen,
- einer Menge s von internen Variablen,
- einer Menge o von Ausgabevariablen,
- einer initialen Konfiguration, beschrieben durch $I \in \mathbb{F}_s$
- einer Transitionsrelation $\rightarrow \subseteq V_s \times V_i \times V_o \times V_s$, beschrieben durch $\Gamma \in \mathbb{F}_{s \cup i \cup o \cup s'}$.

Annahmen:

- Eine interne Variable kann auch in der Ausgabe vorkommen. Aber $i \cap s = \emptyset$ und $i \cap o = \emptyset$
- Komponenten müssen keine Eingabe, Zustand, Ausgabe haben, also i, s oder o können leer sein.

Verhalten von Komponenten (Transitionen)

Komponente $C = \langle \mathbf{i}, \mathbf{s}, \mathbf{o}, I, \Gamma \rangle$

Semantik

Initialisierung:

- Belegung $v_s \in V_s$ ist eine initiale Konfiguration von C wenn $v_s \models I$

Transitionen:

- Tupel $(v_s, v_i, v_o, v'_s) \in \rightarrow$ wenn $v_s \circ v_i \circ v_o[\mathbf{o}'/\mathbf{o}] \circ v'_s[\mathbf{s}'/\mathbf{s}] \models \Gamma$
- Wir schreiben $v_s \xrightarrow{v_i/v_o} v'_s$

Annahmen:

- Variablen aus $\mathbf{s}$ müssen nicht (eindeutig) initialisiert werden
- Komponente kann nicht-deterministisch sein ($\rightarrow$ ist eine Relation).
- Komponenten können immer schalten:
Für jedes Paar (v_s, v_i) existiert min. eine Transition $(v_s, v_i, v_o, v'_s) \in \rightarrow$

Verhalten von Komponenten (Läufe)

Semantik

Komponente $C = \langle \mathbf{i}, \mathbf{s}, \mathbf{o}, I, \Gamma \rangle$

Lauf:

- (Unendliche) Folge von Transitionen $v_s^0 \xrightarrow{v_i^1/v_o^1} v_s^1 \xrightarrow{v_i^2/v_o^2} v_s^2 \xrightarrow{v_i^3/v_o^3} \dots$ mit $v_s^0 \models I$

Verhalten von C :

- Menge aller Läufe

Anmerkung:

- Wir beschäftigen uns hier nur mit endlichen Präfixen von Läufen

Beispiel

Komponente $Delay = \langle \mathbf{i}, \mathbf{s}, \mathbf{o}, I, \Gamma \rangle$ mit

- $\mathbf{i} = \{i_1\}, \mathbf{s} = \{s_1\}, \mathbf{o} = \{o_1\}$ und i_1, s_1, o_1 vom Typ Nat
- $I = (s_1 = 0)$
- $\Gamma = (o'_1 = s_1 \wedge s'_1 = i_1)$

Transitionen in $\rightarrow$ (Beispiele):

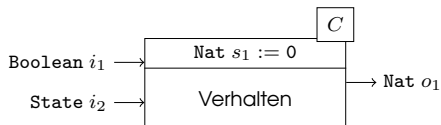
- $\{s_1 \mapsto 0\} \xrightarrow{\{i_1 \mapsto 0\} / \{o_1 \mapsto 0\}} \{s_1 \mapsto 0\}$
- $\{s_1 \mapsto 0\} \xrightarrow{\{i_1 \mapsto 1\} / \{o_1 \mapsto 0\}} \{s_1 \mapsto 1\}$
- $\{s_1 \mapsto 2\} \xrightarrow{\{i_1 \mapsto 3\} / \{o_1 \mapsto 2\}} \{s_1 \mapsto 3\}$
- $\{s_1 \mapsto 3\} \xrightarrow{\{i_1 \mapsto 5\} / \{o_1 \mapsto 3\}} \{s_1 \mapsto 5\}$

Lauf (Anfang):

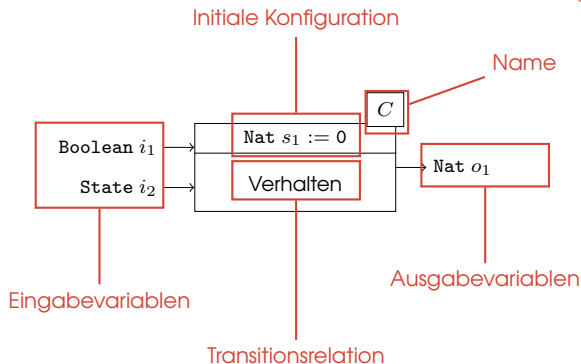
Var.	t_0	t_1	t_2	t_3	t_4
i_1	1	2	5	5	3
s_1	0	1	2	5	5
s'_1	1	2	5	5	3
o'_1	0	1	2	5	5

Konkrete Syntax

Konkrete
Syntax

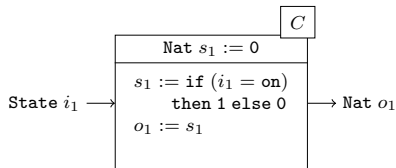


Konkrete Syntax



Kombinatorische Komponente

Konkrete Syntax:



Verhalten:

- Liste von Zuweisungen $x := e$ für $x \in (\mathbf{s} \cup \mathbf{o})$ und $e \in \mathbb{F}_{\mathbf{s} \cup \mathbf{i} \cup \mathbf{s}'}$
- Eine Zuweisung pro Variable
- Sequenzielle Ausführung

Abstrakte Syntax:

$C = \langle \mathbf{i}, \mathbf{s}, \mathbf{o}, \mathbf{I}, \Gamma \rangle$ mit

$\mathbf{i} = \{i_1\}$, $\mathbf{s} = \{s_1\}$, $\mathbf{o} = \{o_1\}$,

$\mathbf{I} = (s_1 = 0)$ und

$\Gamma = (s'_1 = \text{if } (i_1 = \text{on}) \text{ then } 1 \text{ else } 0) \wedge$
 $(o'_1 = s'_1)$

Übersetzung der Zuweisungen:

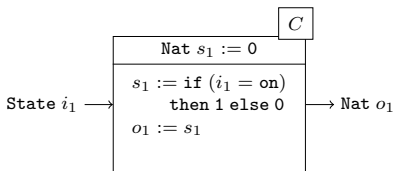
- Menge W : geschriebene Variablen in bereits übersetzten Zuweisungen
- $x := e$ wird zu $x' = w(e)$
- $w(e)$ ersetzt y durch y' in e für $y \in W$
- Γ : Konjunktion aller Zuweisungen

Syntax
abstrakt $\leftrightarrow$ konkret

Beispiel 1: Ausführung von Komponenten

Semantik

Konkrete Syntax:



Transitionen in $\rightarrow$:

- $\{s_1 \mapsto 0\} \xrightarrow{\{i_1 \mapsto \text{off}\} / \{o_1 \mapsto 0\}} \{s_1 \mapsto 0\}$
- $\{s_1 \mapsto 0\} \xrightarrow{\{i_1 \mapsto \text{on}\} / \{o_1 \mapsto 1\}} \{s_1 \mapsto 1\}$
- $\{s_1 \mapsto 1\} \xrightarrow{\{i_1 \mapsto \text{off}\} / \{o_1 \mapsto 0\}} \{s_1 \mapsto 0\}$
- $\{s_1 \mapsto 1\} \xrightarrow{\{i_1 \mapsto \text{on}\} / \{o_1 \mapsto 1\}} \{s_1 \mapsto 1\}$

Abstrakte Syntax:

$C = \langle \mathbf{i}, \mathbf{s}, \mathbf{o}, I, \Gamma \rangle$ mit

$\mathbf{i} = \{i_1\}$, $\mathbf{s} = \{s_1\}$, $\mathbf{o} = \{o_1\}$,

$I = (s_1 = 0)$ und

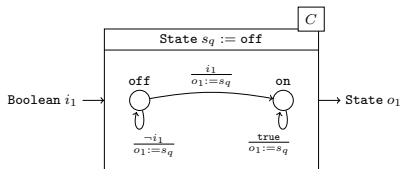
$\Gamma = (s'_1 = \text{if } (i_1 = \text{on}) \text{ then } 1 \text{ else } 0) \wedge (o'_1 = s'_1)$

Lauf (Anfang):

Var.	t_0	t_1	t_2	t_3	t_4
i_1	off	off	on	off	on
s_1	0	0	0	1	0
s'_1	0	0	1	0	1
o'_1	0	0	1	0	1

Zustandsbasierte Komponenten

Konkrete Syntax:



Verhalten:

- Automat, spezielle Variable $s_q \in s$ vom Typ State verwaltet Zustand
- Transitionen $\frac{g}{u}$ mit Bedingung $g \in \mathbb{F}_{s \cup i}$ und Zuweisungsliste u (wie bisher, s_q erste Zuweisung)

Abstrakte Syntax:

$C = \langle i, s, o, I, \Gamma \rangle$ mit

$i = \{i_1\}$, $s = \{s_q\}$, $o = \{o_1\}$,

$I = (s_q = \text{off})$ und

$$\begin{aligned} \Gamma = & (s_q = \text{off} \wedge \neg i_1 \wedge s'_q = \text{off} \wedge o'_1 = s'_q) \vee \\ & (s_q = \text{off} \wedge i_1 \wedge s'_q = \text{on} \wedge o'_1 = s'_q) \vee \\ & (s_q = \text{on} \wedge \text{true} \wedge s'_q = \text{on} \wedge o'_1 = s'_q) \end{aligned}$$

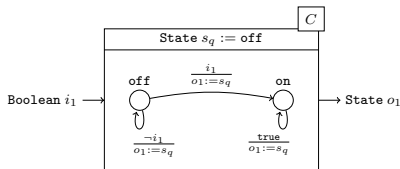
Übersetzung von Transitionen:

- $\frac{g}{u}$ wird zu $(g \wedge \varphi_u)$
- φ_u Übersetzung von u wie für kombinatorische Komponenten
- Γ : Disjunktion aller Transitionen

Syntax
abstrakt $\leftrightarrow$ konkret

Beispiel 2: Ausführung von Komponenten

Konkrete Syntax:



Abstrakte Syntax:

Semantik

$C = \langle \mathbf{i}, \mathbf{s}, \mathbf{o}, I, \Gamma \rangle$ mit

$\mathbf{i} = \{i_1\}$, $\mathbf{s} = \{s_q\}$, $\mathbf{o} = \{o_1\}$,

$I = (s_q = \text{off})$ und

$\Gamma = (s_q = \text{off} \wedge \neg i_1 \wedge s'_q = \text{off} \wedge o'_1 = s'_q) \vee$
 $(s_q = \text{off} \wedge i_1 \wedge s'_q = \text{on} \wedge o'_1 = s'_q) \vee$
 $(s_q = \text{on} \wedge \text{true} \wedge s'_q = \text{on} \wedge o'_1 = s'_q)$

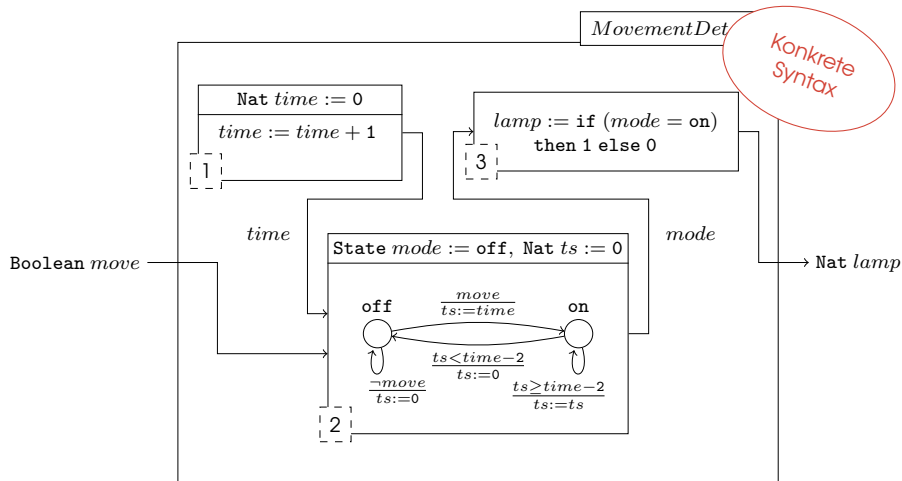
Transitionen in $\rightarrow$:

- $\{s_q \mapsto \text{off}\} \xrightarrow{\{i_1 \mapsto 0\} / \{o_1 \mapsto \text{off}\}} \{s_q \mapsto \text{off}\}$
- $\{s_q \mapsto \text{off}\} \xrightarrow{\{i_1 \mapsto 1\} / \{o_1 \mapsto \text{on}\}} \{s_q \mapsto \text{on}\}$
- $\{s_q \mapsto \text{on}\} \xrightarrow{\{i_1 \mapsto 0\} / \{o_1 \mapsto \text{on}\}} \{s_q \mapsto \text{on}\}$
- $\{s_q \mapsto \text{on}\} \xrightarrow{\{i_1 \mapsto 1\} / \{o_1 \mapsto \text{on}\}} \{s_q \mapsto \text{on}\}$

Lauf (Anfang):

Var.	t_0	t_1	t_2	t_3	t_4
i_1	0	0	1	1	0
s_q	off	off	off	on	on
s'_q	off	off	on	on	on
o'_1	off	off	on	on	on

Komposition von Komponenten (Beispiel)



Komposition von Komponenten (Operationen)

Umbenennung von Variablen:

- $C[x \mapsto y]$: Komponente, die durch Umbenennen der Variable x zu y in C entsteht

Abstrakte
Syntax

Verstecken von Variablen:

- $C \setminus x$: Komponente, die aus C durch Verstecken der Variable x entsteht (x wird aus $\mathbf{i}$ und/oder $\mathbf{o}$ entfernt und $\mathbf{s}$ hinzugefügt)

Sequenzielle Komposition:

- $C_1; C_2$: Komposition von $C_1 = \langle \mathbf{i}_1, \mathbf{s}_1, \mathbf{o}_1, I_1, \Gamma_1 \rangle$ und $C_2 = \langle \mathbf{i}_2, \mathbf{s}_2, \mathbf{o}_2, I_2, \Gamma_2 \rangle$

$C_1; C_2 = \langle \mathbf{i}, \mathbf{s}, \mathbf{o}, I, \Gamma \rangle$ mit

- $\mathbf{i} = \mathbf{i}_1 \cup \mathbf{i}_2$ und $\mathbf{s} = \mathbf{s}_1 \cup \mathbf{s}_2$ und $\mathbf{o} = \mathbf{o}_1 \cup \mathbf{o}_2$,
- $I = I_1 \wedge I_2$, und
- $\Gamma = \Gamma_1 \wedge w(\Gamma_2)$ $w(e)$ ersetzt y durch y' in e für $y \in (\mathbf{s}_1 \cup \mathbf{o}_1)$

Bedingung: $(\mathbf{i}_1 \cup \mathbf{s}_1 \cup \mathbf{o}_1) \cap (\mathbf{s}_2 \cup \mathbf{o}_2) = \emptyset$ und $\mathbf{i}_2 \cap (\mathbf{s}_1 \setminus \mathbf{o}_1) = \emptyset$

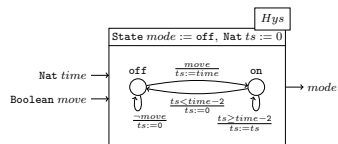
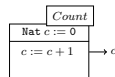
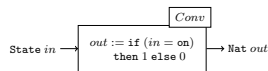
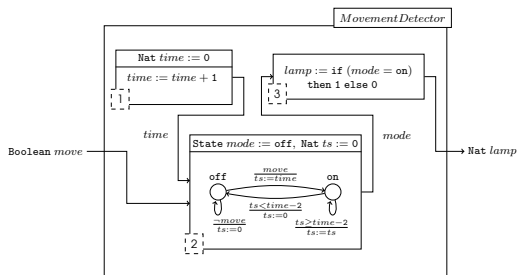
Beispiel: Komposition

Syntax
abstrakt $\leftrightarrow$ konkret

MovementDetector =

$(\text{Count}[c \mapsto \text{time}]; \text{Hys};$

$\text{Conv}[\text{in} \mapsto \text{mode}][\text{out} \mapsto \text{lamp}]) \setminus \text{time} \setminus \text{mode}$



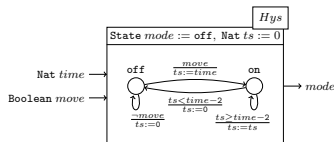
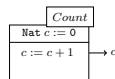
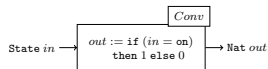
Beispiel Komposition (contd.)

Syntax
abstrakt $\leftrightarrow$ konkret

MovementDetector =
 $\left(\text{Count}[c \mapsto \text{time}]; \text{Hys}; \right.$
 $\left. \text{Conv}[in \mapsto \text{mode}][out \mapsto \text{lamp}] \right) \setminus \text{time} \setminus \text{mode}$

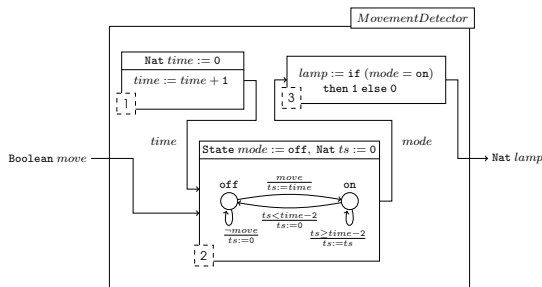
MovementDetector = $\langle i, s, o, I, \Gamma \rangle$ mit:

- $i = \{\text{move}\}, s = \{\text{time}, \text{mode}, \text{ts}\}, o = \{\text{lamp}\}$
- $I = (\text{time} = 0 \wedge \text{mode} = \text{off} \wedge \text{ts} = \text{ts})$
- $\Gamma = (\text{time}' = \text{time} + 1) \wedge$
 $\left((\text{mode} = \text{off} \wedge \neg \text{move} \wedge \text{mode}' = \text{off} \wedge \text{ts} = 0) \vee \right.$
 $(\text{mode} = \text{off} \wedge \text{move} \wedge \text{mode}' = \text{on} \wedge \text{ts}' = \text{time}') \vee$
 $(\text{mode} = \text{on} \wedge \text{ts} \geq \text{time}' - 2 \wedge \text{mode}' = \text{on} \wedge \text{ts}' = \text{ts}) \vee$
 $(\text{mode} = \text{on} \wedge \text{ts} < \text{time}' - 2 \wedge \text{mode}' = \text{off} \wedge \text{ts}' = 0) \Big) \wedge$
 $(\text{lamp}' = \text{if } (\text{mode}' = \text{on}) \text{ then } 1 \text{ else } 0)$



Beispiel Komposition (contd.)

Semantik



$MovementDetector = \langle i, s, o, I, \Gamma \rangle$ mit:

- $i = \{move\}, s = \{time, mode, ts\}, o = \{lamp\}$
- $I = (time = 0 \wedge mode = off \wedge ts = ts)$
- $\Gamma = (time' = time + 1) \wedge$
 $((mode = off \wedge \neg move \wedge mode' = off \wedge ts = 0) \vee$
 $(mode = off \wedge move \wedge mode' = on \wedge ts' = time') \vee$
 $(mode = on \wedge ts \geq time' - 2 \wedge mode' = on \wedge ts' = ts) \vee$
 $(mode = on \wedge ts < time' - 2 \wedge mode' = off \wedge ts' = 0)) \wedge$
 $(lamp' = \text{if } (mode' = on) \text{ then } 1 \text{ else } 0)$

Lauf (Anfang):

Var.	t_0	t_1	t_2	t_3	t_4
<i>move</i>	0	1	0	0	1
<i>time</i>	0	1	2	3	4
<i>mode</i>	off	off	on	on	on
<i>ts</i>	0	0	2	2	2
<i>time'</i>	1	2	3	4	5
<i>mode'</i>	off	on	on	on	off
<i>ts'</i>	0	2	2	2	0
<i>lamp'</i>	0	1	1	1	0

Diskussion: Komponenten und Komposition

Syntax vs. Semantik:

- Komposition: Syntaktisches Werkzeug
- Unterliegende Semantik für alle Komponenten gleich

Top-Down / Bottom-up:

- Architektur und Struktur: Dekomposition (Top-Down)
- Semantik / Verhalten: Komposition (Bottom-Up)

Vereinfachtes Modell:

- Feste Reihenfolgenfolge in Komponente
- Nur Datenfluss, kein Kontrollfluss
- Keine Ereignisse / Nachrichten / komplexe Datentypen

In der Praxis:

- Modellbasierte Softwareentwicklung in der Automobilbranche und in der Luftfahrt
- Regelungstechnische Systeme

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Modellierung des Verhaltens von Komponenten
- 4. Analyse des Verhaltens von Komponenten**
5. Literatur und Standards

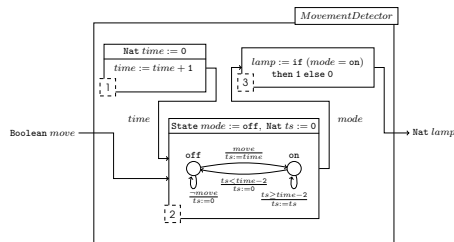
Analyse des Verhaltens von Komponenten

Semantische Korrektheit:

- Erfüllt das Modell funktionale Anforderungen?

In diesem Abschnitt:

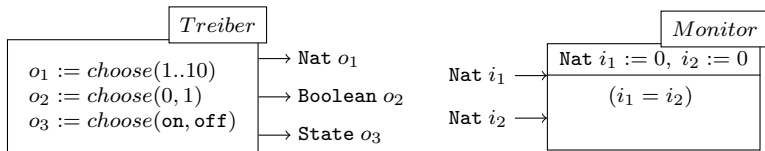
- Analyse von Modellen
- Bounded Model Checking
- Induktive Invarianten



Beispiel:

- Frage: Haben wir korrekt modelliert?
- Z.B. Geht das Licht an, wenn sich etwas bewegt?

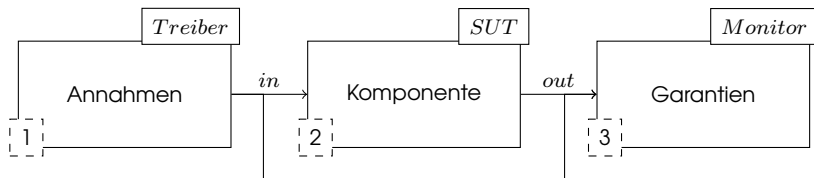
Testen mit Treiber und Monitor



$$(o'_1 \geq 1 \wedge o'_1 \leq 10) \wedge (o'_2 \vee \neg o'_2) \wedge (o'_3 = \text{on} \vee o'_3 = \text{off})$$

- **Treiber** liefern Werte für alle Eingabevariablen der zu testenden Komponente (Annahmen über Wahl der Werte)
- **Monitore** beschreiben zu prüfende Eigenschaften und machen Ausgaben der Komponente zu Zustand

Open-Loop Analyse



- SUT = System unter Test (zu testende Komponente)
- Open-Loop: Treiber und Komponente in Reihe, kein Feedback
- Annahmen über Umgebung nur durch Eingaben
- Test-Aufbau in dieser Veranstaltung

Transitionssysteme

- Kombination von Komponente, Treiber und Monitor hat keine Eingaben und Ausgaben
- Ohne Eingaben und Ausgaben können wir unsere Definition vereinfachen:

Definition (Transitionssystem)

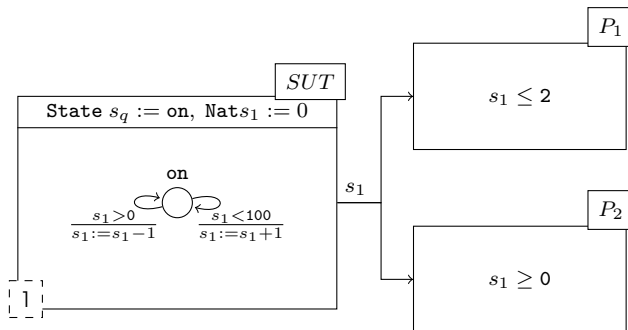
Ein Transitionssystem ist ein Tupel $\langle s, I, \Gamma \rangle$ mit

- einer Menge s von getypten Variablen, die eine Menge von Zuständen V_s (mögliche Belegungen von s) aufspannen,
- einem Booleschen Ausdruck $I \in \mathbb{F}_s$ über s der die initialen Zustände $\llbracket I \rrbracket \subseteq V_s$ beschreibt und
- einem Booleschen Ausdruck $\Gamma \in \mathbb{F}_{s \cup s'}$ der die Transitionsrelation $\llbracket \Gamma \rrbracket \subseteq V_s \times V_s$ beschreibt.

Transitionen:

- $(v_s, v'_s) \in \llbracket \Gamma \rrbracket$ wenn $v_s \circ v'_s[s'/s] \models \Gamma$

Beispiel



- SUT nicht-deterministisch; erhöht oder verringert s_1 um 1 in Grenzen 0 bis 100
- Zwei Monitore für P_1 (gilt nicht immer) und P_2 (gilt immer)

Beispiel (contd.)

Sei $\langle s, I, \Gamma \rangle$ mit $s = \{s_q, s_1\}$ mit s_q vom Typ **State** und s_1 vom Typ **Nat**.

Initialisierung:

Für $I = (s_q = \text{on} \wedge s_1 = 0)$ ist $\llbracket I \rrbracket = \{s_q \mapsto \text{on}, s_1 \mapsto 0\}$.

Transitionen:

Für $\Gamma = ((s_q = \text{on}) \wedge (s_1 < 100) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 + 1)) \vee$
 $((s_q = \text{on}) \wedge (s_1 > 0) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 - 1))$

Dann ist $\llbracket \Gamma \rrbracket = \{ (\{s_q \mapsto \text{on}, s_1 \mapsto 0\}, \{s_q \mapsto \text{on}, s_1 \mapsto 1\}),$
 $\dots,$
 $(\{s_q \mapsto \text{on}, s_1 \mapsto 1\}, \{s_q \mapsto \text{on}, s_1 \mapsto 0\}),$
 $(\{s_q \mapsto \text{on}, s_1 \mapsto -2\}, \{s_q \mapsto \text{on}, s_1 \mapsto -1\}),$
 $\dots \}$

Erreichbare Belegungen

Definition (Erreichbare Belegungen)

In $T = \langle s, I, \Gamma \rangle$ ist eine Belegung $v_s \in V_s$ erreichbar wenn es eine Sequenz von Belegungen $v_0, v_1, \dots, v_k$ in V_s gibt, so dass

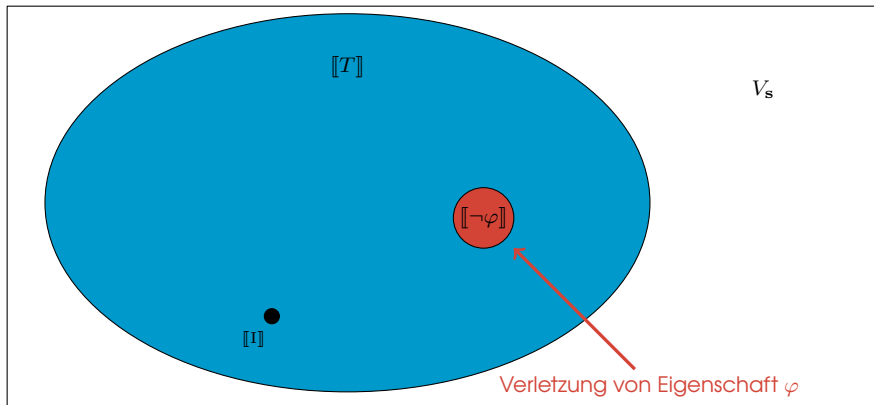
$$v_0 \in \llbracket I \rrbracket \quad \text{und} \quad (v_{i-1}, v_i) \in \llbracket \Gamma \rrbracket \quad \text{für} \quad 1 \leq i \leq k \quad \text{und} \quad v_k = v_s.$$

Wir schreiben $\llbracket T \rrbracket$ für die Menge aller **erreichbaren Belegungen** von T .

Beispiel:

- Sei $T = \langle s, I, \Gamma \rangle$ wie auf der vorigen Folie
- Dann ist $(\{s_q \mapsto \text{on}, s_1 \mapsto -2\}, \{s_q \mapsto \text{on}, s_1 \mapsto -1\}) \in \llbracket \Gamma \rrbracket$
- Aber $\llbracket T \rrbracket = \{ \{s_q \mapsto \text{on}, s_1 \mapsto 0\}, \{s_q \mapsto \text{on}, s_1 \mapsto 1\}, \dots, \{s_q \mapsto \text{on}, s_1 \mapsto 100\} \}$

Zustandsraum und Eigenschaften



- V_s und $\llbracket T \rrbracket$ generell nicht endlich

Mögliche Ergebnisse einer Analyse

Frage:

Gilt Eigenschaft P für Transitionssystem T bzw.
gilt P allen erreichbaren Belegungen in $\llbracket T \rrbracket$?

Mögliche Antworten mit Fehlern:

- **True Positive (TP)** Antwort: P kann verletzt werden. Und P wird tatsächlich verletzt. Ein Fehler liegt vor und wird gefunden
- **True Negative (TN)** Antwort: Keine Verletzung von P gefunden. Und P wird tatsächlich nicht verletzt. Es liegt kein Fehler vor und es wird auch keiner berichtet
- **False Positive (FP)** Antwort: P kann verletzt werden. Aber: P wird tatsächlich nicht verletzt. Es wird ein Fehler berichtet, obwohl kein Fehler vorliegt
- **False Negative (FN)** Antwort: Keine Verletzung von P gefunden. Aber P wird verletzt. Es wird kein Fehler berichtet, obwohl ein Fehler vorliegt

Bounded Model Checking (BMC)



- Model Checking = Systematisches Explorieren der Zustände eines Systems zur Überprüfung von Eigenschaften
- Unendlich viele Zustände $\Rightarrow$ Vollständige Exploration evtl. unmöglich

k -Sicherheit

- Für $T = \langle s, I, \Gamma \rangle$ sei $\Gamma^{i,j} = \Gamma[s^i/s][s^j/s']$
- Für Eigenschaft P sei $P^k = P[s^k/s]$

Definition (k -Sicherheit)

Ein Transitionssystem $T = \langle s, I, \Gamma \rangle$ ist k -sicher für eine Eigenschaft P , wenn P für min. k Schritte gilt:

$$(I[s^0/s] \wedge \Gamma^{0,1} \wedge \dots \wedge \Gamma^{k-1,k}) \models (P^0 \wedge \dots \wedge P^k).$$

- 0-Sicherheit: $I[s^0/s] \models P^0$
- 1-Sicherheit: $(I[s^0/s] \wedge \Gamma^{0,1}) \models (P^0 \wedge P^1)$
- ...

Wir schreiben $a \models b$ für a impliziert b

Bounded Model Checking

Wdh. Implikation:

- Zur Erinnerung: Wir testen $(a \models b)$, indem wir nach einem Modell für $(a \wedge \neg b)$ suchen (vgl. Folien SWK-2 Anforderungen und Spezifikation).
- Wenn es kein Modell für $(a \wedge \neg b)$ gibt, dann gilt $(a \models b)$

Test für k -Sicherheit:

- Wir testen k -Sicherheit von $T = \langle s, I, \Gamma \rangle$ bzgl. P indem wir nach einem Modell suchen für

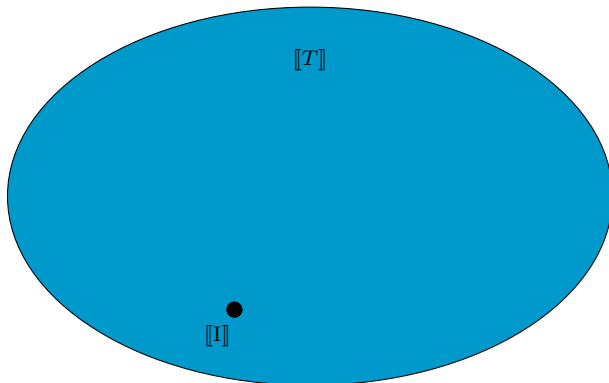
$$I[s^0/s] \wedge \Gamma^{0,1} \wedge \dots \wedge \Gamma^{k-1,k} \wedge (\neg P^0 \vee \dots \vee \neg P^k).$$

Definition (Bounded Model Checking)

Für Transitionssystem $T = \langle s, I, \Gamma \rangle$ und $k = 0$:

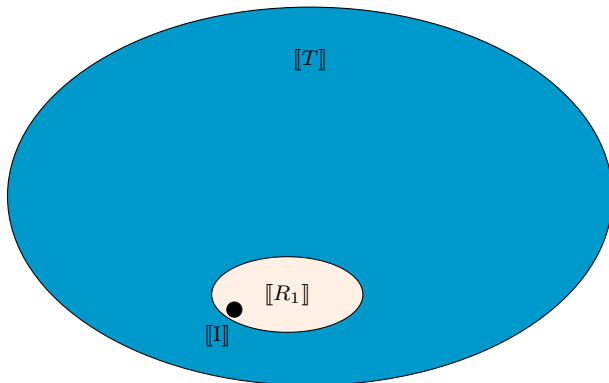
- ① Wenn Test für k -Sicherheit von T erfüllbar ist,
- ② dann Fehler gefunden! Stop mit Modell als Gegenbeispiel.
- ③ Sonst $k = k + 1$.
- ④ Gehe zu (1).

Intuition: Bounded Model Checking



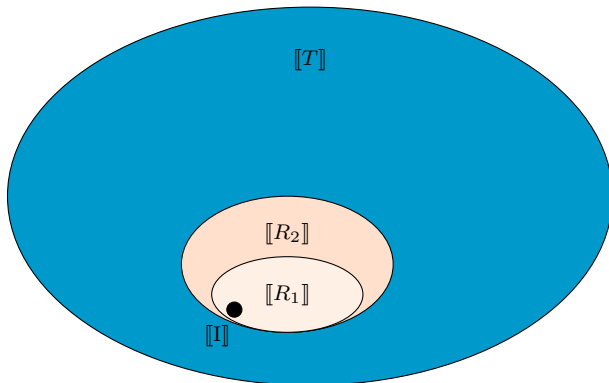
- $\llbracket R_i \rrbracket$: Menge der Belegungen, die in max. i Schritten erreicht werden.
- Es gilt $\llbracket R_i \rrbracket \subseteq \llbracket R_{i+1} \rrbracket$
- Es gibt keine Möglichkeit festzustellen, wann $\llbracket T \rrbracket \subseteq \llbracket R_i \rrbracket$ da wir keine logische Formel haben, die $\llbracket T \rrbracket$ beschreibt

Intuition: Bounded Model Checking



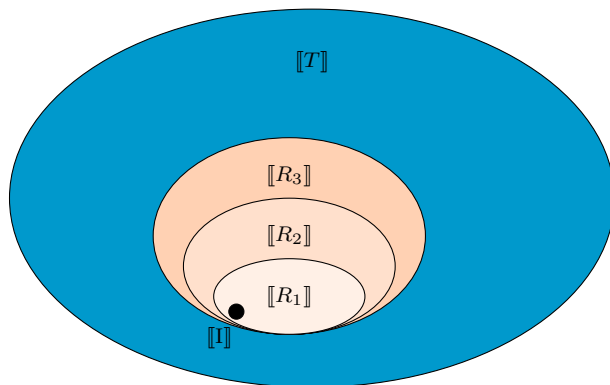
- $\llbracket R_i \rrbracket$: Menge der Belegungen, die in max. i Schritten erreicht werden.
- Es gilt $\llbracket R_i \rrbracket \subseteq \llbracket R_{i+1} \rrbracket$
- Es gibt keine Möglichkeit festzustellen, wann $\llbracket T \rrbracket \subseteq \llbracket R_i \rrbracket$ da wir keine logische Formel haben, die $\llbracket T \rrbracket$ beschreibt

Intuition: Bounded Model Checking



- $\llbracket R_i \rrbracket$: Menge der Belegungen, die in max. i Schritten erreicht werden.
- Es gilt $\llbracket R_i \rrbracket \subseteq \llbracket R_{i+1} \rrbracket$
- Es gibt keine Möglichkeit festzustellen, wann $\llbracket T \rrbracket \subseteq \llbracket R_i \rrbracket$ da wir keine logische Formel haben, die $\llbracket T \rrbracket$ beschreibt

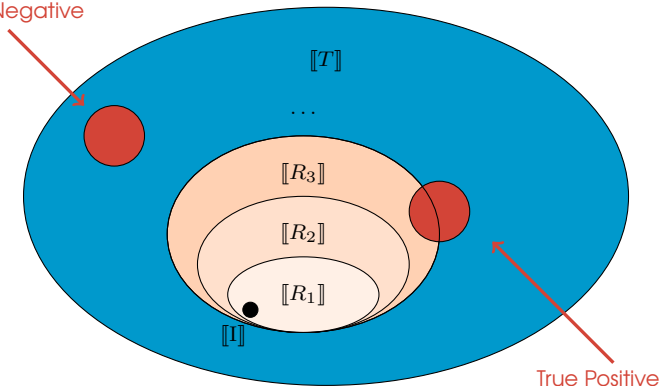
Intuition: Bounded Model Checking



- $\llbracket R_i \rrbracket$: Menge der Belegungen, die in max. i Schritten erreicht werden.
- Es gilt $\llbracket R_i \rrbracket \subseteq \llbracket R_{i+1} \rrbracket$
- Es gibt keine Möglichkeit festzustellen, wann $\llbracket T \rrbracket \subseteq \llbracket R_i \rrbracket$ da wir keine logische Formel haben, die $\llbracket T \rrbracket$ beschreibt

Intuition: Bounded Model Checking

False Negative



True Positive

- $\llbracket R_i \rrbracket$: Menge der Belegungen, die in max. i Schritten erreicht werden.
- Es gilt $\llbracket R_i \rrbracket \subseteq \llbracket R_{i+1} \rrbracket$
- Es gibt keine Möglichkeit festzustellen, wann $\llbracket T \rrbracket \subseteq \llbracket R_i \rrbracket$ da wir keine logische Formel haben, die $\llbracket T \rrbracket$ beschreibt

Beispiel: Bounded Model Checking

Sei $\langle s, I, \Gamma \rangle$ mit $s = \{s_q, s_1\}$ wie bisher und $I = (s_q = \text{on} \wedge s_1 = 0)$

Transitionen:

$$\begin{aligned} \text{Für } \Gamma = & ((s_q = \text{on}) \wedge (s_1 < 100) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 + 1)) \vee \\ & ((s_q = \text{on}) \wedge (s_1 > 0) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 - 1)) \end{aligned}$$

$$\begin{aligned} \text{ist } \Gamma^{0,1} = & ((s_q^0 = \text{on}) \wedge (s_1^0 < 100) \wedge (s_q^1 = \text{on}) \wedge (s_1^1 = s_1^0 + 1)) \vee \\ & ((s_q^0 = \text{on}) \wedge (s_1^0 > 0) \wedge (s_q^1 = \text{on}) \wedge (s_1^1 = s_1^0 - 1)) \end{aligned}$$

Test für 1-Sicherheit bzgl. P_1 ($s_1 \leq 2$):

$$\begin{aligned} & (s_q^0 = \text{on} \wedge s_1^0 = 0) \wedge & // I[s^0/s] \\ & (((s_q^0 = \text{on}) \wedge (s_1^0 < 100) \wedge (s_q^1 = \text{on}) \wedge (s_1^1 = s_1^0 + 1)) \vee & // \Gamma^{0,1} \\ & ((s_q^0 = \text{on}) \wedge (s_1^0 > 0) \wedge (s_q^1 = \text{on}) \wedge (s_1^1 = s_1^0 - 1))) \wedge \\ & (\neg(s_1^0 \leq 2) \vee \neg(s_1^1 \leq 2)) & // \neg P^0 \vee \neg P^1 \end{aligned}$$

T ist 2-sicher aber nicht 3-sicher für P_1 !

Z3 Beispiel: <https://rise4fun.com/Z3/IC5D>

Invarianten



- Manchmal wollen wir **beweisen**, dass eine Eigenschaft immer gilt
- Ein Invariante ist eine Eigenschaft, die in jedem Zustand eines Systems gilt
- Typischerweise über-approximiert eine Invariante $\llbracket T \rrbracket$ ist dafür aber einfach logisch ausdrückbar

Induktive Invarianten

Definition (Induktive Invariante)

Eine Eigenschaft P ist eine induktive Invariante von $T = \langle s, I, \Gamma \rangle$ wenn gilt dass:

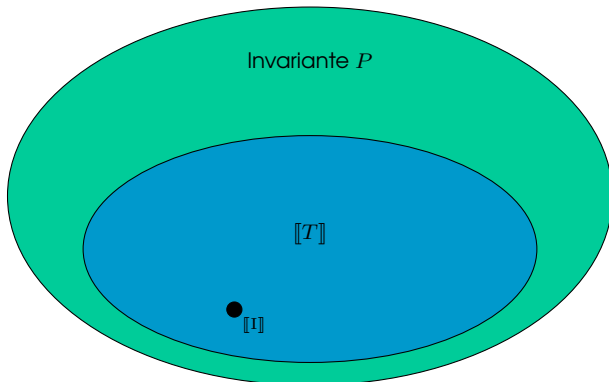
- ① jede Belegung in $v_s \in \llbracket I \rrbracket$ die Eigenschaft P erfüllt ($v_s \models P$), und
- ② falls eine Belegung v_s die Eigenschaft P erfüllt und (v_s, v'_s) in $\llbracket \Gamma \rrbracket$ ist, dann erfüllt auch v'_s die Eigenschaft P .

Induktionsbeweis:

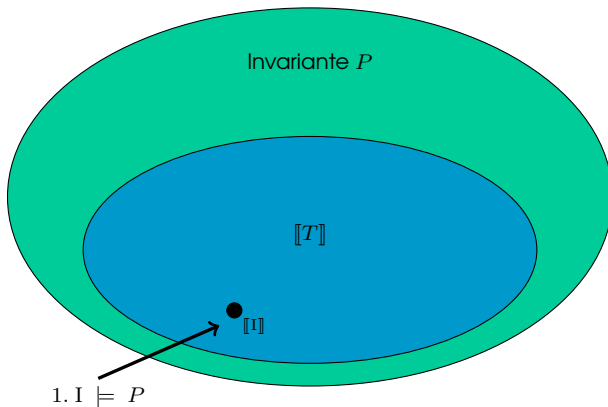
1. Basis: $I \models P$

2. Schritt: $P \wedge \Gamma \models P[s'/s]$

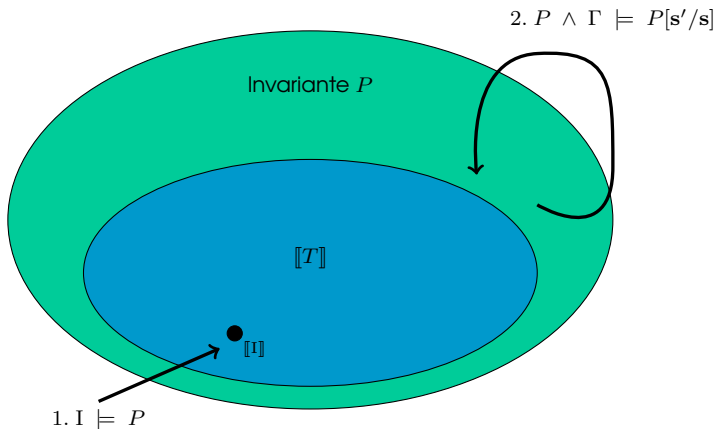
Intuition: Induktive Invarianten



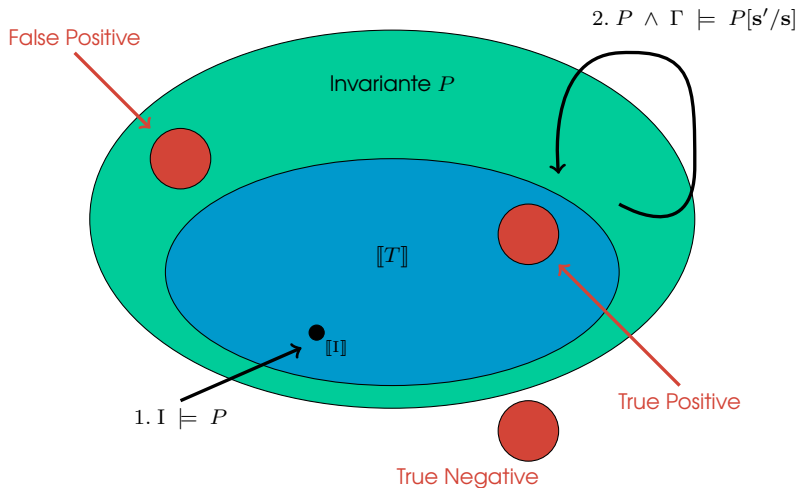
Intuition: Induktive Invarianten



Intuition: Induktive Invarianten



Intuition: Induktive Invarianten



Beispiel: Induktive Invariante

Sei $\langle s, I, \Gamma \rangle$ mit $s = \{s_q, s_1\}$ wie bisher mit $I = (s_q = \text{on} \wedge s_1 = 0)$ und

$$\Gamma = ((s_q = \text{on}) \wedge (s_1 < 100) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 + 1)) \vee \\ ((s_q = \text{on}) \wedge (s_1 > 0) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 - 1))$$

Wir beweisen $P_2 (s_1 \geq 0)$:

Beispiel: Induktive Invariante

Sei $\langle s, I, \Gamma \rangle$ mit $s = \{s_q, s_1\}$ wie bisher mit $I = (s_q = \text{on} \wedge s_1 = 0)$ und

$$\Gamma = ((s_q = \text{on}) \wedge (s_1 < 100) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 + 1)) \vee \\ ((s_q = \text{on}) \wedge (s_1 > 0) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 - 1))$$

Wir beweisen $P_2 (s_1 \geq 0)$:

1. **Basis:** $I \models P$

$$(s_q = \text{on} \wedge s_1 = 0) \models (s_1 \geq 0) \quad \checkmark$$

Beispiel: Induktive Invariante

Sei $\langle s, I, \Gamma \rangle$ mit $s = \{s_q, s_1\}$ wie bisher mit $I = (s_q = \text{on} \wedge s_1 = 0)$ und

$$\Gamma = ((s_q = \text{on}) \wedge (s_1 < 100) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 + 1)) \vee \\ ((s_q = \text{on}) \wedge (s_1 > 0) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 - 1))$$

Wir beweisen $P_2 (s_1 \geq 0)$:

1. Basis: $I \models P$

$$(s_q = \text{on} \wedge s_1 = 0) \models (s_1 \geq 0) \quad \checkmark$$

2. Schritt: $P \wedge \Gamma \models P[s'/s]$

$$(s_1 \geq 0) \wedge (((s_q = \text{on}) \wedge (s_1 < 100) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 + 1)) \vee \\ ((s_q = \text{on}) \wedge (s_1 > 0) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 - 1))) \models (s'_1 \geq 0) \quad \checkmark$$

Beispiel: Induktive Invariante (SMT)

Sei $\langle s, I, \Gamma \rangle$ und P wie eben

1. Basis: $I \models P$

$$(s_q = \text{on} \wedge s_1 = 0) \wedge \neg(s_1 \geq 0) \quad \text{unsat}$$

2. Schritt: $P \wedge \Gamma \models P[s'/s]$

$$(s_1 \geq 0) \wedge \left(((s_q = \text{on}) \wedge (s_1 < 100) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 + 1)) \vee \right. \\ \left. ((s_q = \text{on}) \wedge (s_1 > 0) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 - 1)) \right) \wedge \neg(s'_1 \geq 0) \quad \text{unsat}$$

Wenn es kein Modell für $(a \wedge \neg b)$ gibt, dann gilt $(a \models b)$

Z3 Beispiel: <https://rise4fun.com/Z3/0LXA4>

Diskussion: BMC und Invarianten

Bounded Model Checking

- Iteratives Abrollen des Transitionssystems
- Keine Möglichkeit festzustellen, wann/ob alle erreichbaren Belegungen untersucht

Fehler:

- False Positives: Nein, Modell für BMC Formel ist direkt ein Zeuge für Verletzung der Eigenschaft
- False Negatives: Möglich, u.U. nicht genug Zeit / Ressourcen um Transitionssystem tief genug abzurollen

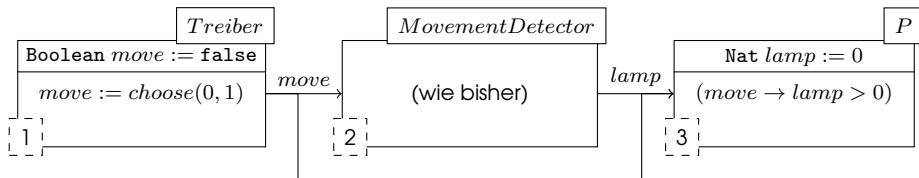
Induktive Invarianten

- Induktionsbeweis
- Über-approximiert $\llbracket T \rrbracket$

Fehler:

- False Positives: Möglich, durch Über-Approximierung
- False Negatives: Nein, da Beweis

Beispiel: Remote Home Control

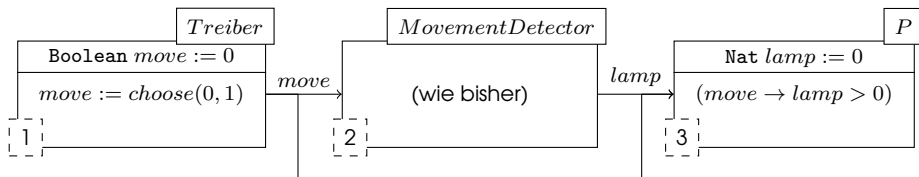


Wir missbrauchen die Notation etwas: eigentlich müsste die Initialisierung von *lamp* zu *MovementDetector* gehören.

Transitionssystem $Treiber; MovementDetector; P = \langle s, I, \Gamma \rangle$ mit:

- $s = \{move, time, mode, ts, lamp\}$
- $I = (\neg move \wedge time = 0 \wedge mode = off \wedge ts = 0 \wedge lamp = 0)$
- $\Gamma = (move' \vee \neg move') \wedge (time' = time + 1) \wedge$
 $((mode = off \wedge \neg move' \wedge mode' = off \wedge ts = 0) \vee$
 $(mode = off \wedge move' \wedge mode' = on \wedge ts' = time') \vee$
 $(mode = on \wedge ts \geq time' - 2 \wedge mode' = on \wedge ts' = ts) \vee$
 $(mode = on \wedge ts < time' - 2 \wedge mode' = off \wedge ts' = 0)) \wedge$
 $(lamp' = \text{if } (mode' = on) \text{ then } 1 \text{ else } 0)$

Beispiel: Remote Home Control



Wir missbrauchen die Notation etwas: eigentlich müsste die Initialisierung von `lamp` zu `MovementDetector` gehören.

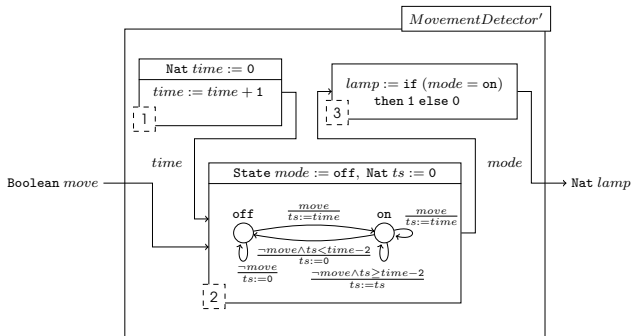
■ Gewünschte Eigenschaft:

Wenn Bewegung dann Licht an ($move \rightarrow lamp > 0$)

■ BMC:

Eigenschaft ist nicht 3-sicher (Licht geht nach Nachlauf erst einmal aus)!

Beispiel: Remote Home Control



■ Anpassung:

MovementDetector': Extra Transition im Automaten für Fall *mode = on* und *move*

■ Invariante:

$(move \rightarrow lamp > 0)$ beweisbar auf *MovementDetector'*

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Modellierung des Verhaltens von Komponenten
4. Analyse des Verhaltens von Komponenten
5. Literatur und Standards

- Ranjit Jhala and Rupak Majumdar: Software Model Checking ACM Computing Surveys (CSUR) Volume 41 Issue 4, October 2009.
- Rajeev Alur: Principles of Cyber-Physical Systems. Mit Press, 2015.
- Thomas Kühne: Matters of (Meta-) Modeling, STTT (2006) 5: 369. Springer, 2006.
- Clark Barrett, Roberto Sebastiani, Sanjit A. Seshia and Cesare Tinelli: Satisfiability Modulo Theories. In: Armin Biere, Marijn Heule, Hans van Maaren and Toby Walsch. Handbook of satisfiability. IOS Press, 2008.

Standards

- **ISO/IEC 19508:2014** Information technology - Meta Object Facility (MOF)
- **OMG MDA** Model Driven Architecture
- **OMG 2007** OMG: Unified Modeling Language: Superstructure, Version 2.1.1. OMG document formal/2007-02-05.